# Is Folding Worth the Storage?
# Curved and Shrinking Space-Time Gaussians for Video

*Name: Chen Hongshan*
*Student ID: A0311136W*

## 1. Summary

This project tests whether the two "folding" extensions of **space-time Gaussians** proposed by *VeGaS*, a curved (quadratic) trajectory and a shrinking of each slice as it fades, still improve video fitting once storage is held equal. I plan to

- extend the space-time Gaussian fit of Unit 16 and compare **four models in a 2×2 ablation** at four storage budgets on three UVG sequences with different kinds of motion: *Jockey* (a galloping horse filmed by a panning camera), *ShakeNDry* (a dog shaking water off its fur, filmed by a nearly still camera) and *YachtRide* (a boat filmed by a camera that follows it smoothly);
- apply temporal **subsampling**, keeping only every third frame, which enlarges the motion between consecutive frames;
- check whether any gain comes from the moving regions or from the background, and analyse why each extension helps or does not.

## 2. Research question

At an equal number of stored parameters, does adding a quadratic acceleration term, a fade-dependent shrinking of the slice covariance, or both, to the space-time Gaussians of Unit 16 raise the mean PSNR of 10-frame clips from three UVG sequences with different motion (*Jockey*, *ShakeNDry* and *YachtRide*, at 480×270)? Does the gain grow when the motion is faster, simulated by temporally subsampling each clip to every third frame? If either extension helps, is the gain concentrated in the regions of non-uniform motion rather than in the background?

## 3. Motivation and relation to the course

Unit 16 represents a video by Gaussians in space and time. Slicing one at time t gives a 2D Gaussian whose centre moves at a constant velocity, whose shape is constant, and whose weight fades away from its peak time. Each Gaussian costs 12 stored numbers however many frames it spans, but a patch that swings or changes size, such as the leg of a galloping horse, which swings forward and back and changes its apparent length within a few frames, needs several short-lived Gaussians.

*VeGaS* (Smolak-Dyżewska et al., 2024) generalises this with Folded-Gaussians. The slice centre follows a polynomial in time (degree 7 by default), and the slice covariance is scaled by the time likelihood, so a Gaussian also shrinks as it fades. The paper reports higher reconstruction PSNR on DAVIS than NeRV-family models and VGR, and its main goal is realistic video editing.

The paper, however, does not show that folding is worth its storage. Its ablation (Table 3 of the

paper) uses *Big Buck Bunny*, an open-source animated short film of which the paper uses 132 frames at 720×1280, and varies the polynomial degree from 1 to 9: at batch size 3, degree 1 reaches 38.15 dB with 0.57M Gaussians and degree 7 reaches 38.53 dB with 0.62M. A higher degree adds coefficients to every Gaussian and the final counts differ, so the gain cannot be attributed to the curved trajectory itself. Shrinking is never ablated separately, no model size is reported, and all runs use hundreds of thousands of Gaussians.

The answer is also not obvious from the course. Units 14 and 15 judge a representation by its quality at a given size. The acceleration term costs 2 extra numbers for every Gaussian, including the background Gaussians, which drift with the near-uniform camera pan described in Unit 16 and do not need to curve. Under a fixed budget, fewer Gaussians that can curve therefore compete with more Gaussians that move only in straight lines, and either could win.

Furthermore, whether the extensions pay off may also depend on the kind of motion being presented in the video. UVG sequences are captured at a high frame rate (120 fps for Jockey), so 10 consecutive frames span less than a tenth of a second, and most motion within them is close to linear. A result on one such clip could only show that the clip has nothing to curve. I therefore plan to test three sequences chosen for different kinds of motion, and I temporally subsample some clips to every third frame to simulate faster motion without changing the content or the number of frames.

The question builds on five units of the course: Unit 1 (conditioning of a Gaussian, Cholesky factors), Unit 5 (gradients of an image loss), Unit 6 (PSNR, SSIM and the evaluation protocol), Unit 14 (equal-budget comparisons and rate–distortion) and Unit 16 (the representation being extended).

## 4. Problem formulation

I plan to compare four models that differ only in how a space-time Gaussian is sliced at time t. All four render a frame as the accumulated sum of Unit 16, clamped to [0, 1]:

$$C_t(p) = \Sigma_i\, w_i(t) \cdot c_i \cdot \exp\!\left(-\tfrac{1}{2}\,(p-\mu_i(t))^T\, \Sigma_i(t)^{-1}\,(p-\mu_i(t))\right), \quad w_i(t) = \exp\!\left(-(t-t_{0,i})^2 / (2\,\sigma_{t,i}^2)\right)$$

Each Gaussian stores its centre μ (2 numbers), peak time t0 (1), duration σt (1), velocity v (2), the Cholesky factor L of its slice covariance (3) and colour c (3), as in the Unit 16 example. Writing $\tau = t - t0$:

| Model | Slice centre μ(t) | Slice covariance Σ(t) | Numbers per Gaussian |
|---|---|---|---|
| M0 baseline (Unit 16) | $\mu + v\tau$ | $L\,L^T$ | 12 |
| M1 acceleration | $\mu + v\tau + a\tau^2$ | $L\,L^T$ | 14 |
| M2 shrinking | $\mu + v\tau$ | $\max(w(t), s_{min}) \cdot L\,L^T$ | 12 |
| M3 both | $\mu + v\tau + a\tau^2$ | $\max(w(t), s_{min}) \cdot L\,L^T$ | 14 |

The acceleration $a$ adds 2 numbers per Gaussian. Shrinking adds none; the floor $s_{min}$ keeps the covariance invertible as the weight approaches zero.

**Equal storage.**
For a budget of B stored numbers, M0 and M2 use $\lfloor B/12 \rfloor$ Gaussians and M1 and M3 use $\lfloor B/14 \rfloor$. All parameters are float32 and none are quantised, so every number takes 4 bytes and an equal count of stored numbers is also an equal file size. I report sizes in both numbers and megabytes. For each 10-frame clip, $B \in \{3{,}000; 6{,}000; 12{,}000; 24{,}000\}$, which is 250, 500, 1,000

and 2,000 Gaussians of M0, the counts used in the Unit 16 example. For the 30-frame Jockey clip the budgets are tripled, so each frame receives the same budget.

**Temporal subsampling (frame stride).**
A clip with stride s consists of frames k, k + s, …, k + 9s of the original sequence, re-indexed as t = 0, …, 9. I use s ∈ {1, 3}. A larger stride gives larger and more curved motion between consecutive frames, while the content, the number of frames and the budget stay the same.

**Experimental design.**
The table lists every factor of the experiment. Each model is fitted once for every combination of budget, sequence, stride and seed, and M1, M2 and M3 are always compared with M0 under the same combination.

| Factor | Values | Count | Role |
|---|---|---|---|
| Model | M0, M1, M2, M3 | 4 | What is compared |
| Storage budget B (stored numbers) | 3,000; 6,000; 12,000; 24,000 | 4 | Test condition: model size |
| Sequence | Jockey, ShakeNDry, YachtRide (Section 5) | 3 | Test condition: kind of motion |
| Stride s | 1, 3 | 2 | Test condition: speed of motion |
| Random seed | 3 seeds | 3 | Measures run-to-run variation |
| Total, 10-frame clips | 4 × 4 × 3 × 2 × 3 | 288 fits | Main experiment |
| 30-frame Jockey clip (s = 1, budgets tripled) | 4 models × 4 budgets × 3 seeds | 48 fits | Tests a longer clip |

*Note: The sequence and the stride are test conditions, not parts of a model: they change how much curved motion the video contains, so I can see whether the gain of each extension rises with it.*

**Held fixed.**
The renderer, the loss (mean squared error over all frames, fitted jointly), the Adam optimiser and learning rates of shared parameters, the iteration count, the 480×270 resolution, the number of frames per clip, and the random seed and initialisation of shared parameters. The acceleration starts at a = 0, so M1 is identical to M0 at initialisation. There is no densification or pruning, so every model keeps exactly its budgeted number of Gaussians.

**Swept hyperparameters**,
chosen on a validation clip (described in Section 5): the shrink floor smin ∈ {0.05, 0.1, 0.2} for M2 and M3, and the learning rate of a ∈ {0.3, 1, 3} × the learning rate of v for M1 and M3. The other factors in the design table are not tuned; every combination of them is run.

## 5. Planned implementation

The two extensions, acceleration (M1) and shrinking (M2), change only the slicing step; the renderer is the same image renderer for all four models. I therefore write the slicing step myself and reuse the course's 2D renderer and its hand-written gradients.

**What exists already**

- The Unit 16 formulation of space-time Gaussians and the course program gaussian_image_video.py. It provides a Warp renderer that sums 2D Gaussians, a hand-written backward pass for each Gaussian's centre, Cholesky factor and colour, an Adam fitting loop, and a finite-difference gradient check. It fits images and predicts video frame by frame, but has no space-time Gaussians.

- The GaussianImage paper and code (Zhang et al., ECCV 2024), the source of the accumulated-sum rendering; its CUDA rasteriser is a fallback only.
- The UVG sequences Jockey, ShakeNDry and YachtRide (Mercat et al., 2020).
- OpenCV's Farnebäck optical flow, used only for measuring motion and building motion masks.

**What I will write**

- A slicing step for M0–M3 that maps each Gaussian and a time t to its 2D centre, Cholesky factor and time-weighted colour, which are passed to the course's render and gradient kernels.
- The joint fitting loop over all frames of a clip, with logging of loss, PSNR and time.
- The chain rule from the per-frame 2D gradients that the course kernels return to the space-time parameters (μ, v, a, t0, σt, L, c), summed over frames, including the derivatives of the acceleration and shrinking terms. I will check it against finite differences by extending the course's gradient check.
- A loader for the three sequences that resizes to 480×270 and samples frames with stride s.
- A motion measurement for each clip from optical flow: the mean displacement per frame (speed) and the mean change of displacement between consecutive frames (acceleration).
- A synthetic test clip: 10 frames at 480×270 with a few coloured blobs moving along known parabolic and back-and-forth paths over a static background.
- Evaluation scripts for per-frame, per-region and per-seed metrics, plots and crops. For Jockey, hand-drawn horse masks for frames 0–29, which cover all Jockey test clips. For ShakeNDry and YachtRide, motion masks from optical flow: pixels whose flow differs from the median (camera) flow by more than a threshold. I will check the flow masks against the hand-drawn masks on Jockey.
- Speed fallback: if the Warp renderer is too slow for all fits, I will switch to the GaussianImage CUDA rasteriser, which computes the same accumulated sum.

**Data.**

All sequences are reduced to 480×270, as in Unit 16. The three sequences are chosen for different motion:

| **Sequence** | **Motion** | **Role in the study** |
|---|---|---|
| *Jockey* | Camera pans; horse gallops, legs swing back and forth | Main case, comparable with the Unit 16 notes |
| *ShakeNDry* | Camera nearly still; dog shakes its body back and forth | Strongly curved motion: where acceleration should help most |
| *YachtRide* | Camera follows a boat; smooth, near-rigid motion | Control: acceleration should not help here |

I assigned these roles after watching the sequences. Before the main runs, I will confirm them with the motion measurement above; if a sequence's motion does not match its role, I will replace it with another UVG sequence that does (for example *ReadySetGo* or *HoneyBee*).

- Test clips: for each sequence, frames 0–9 (s = 1) and frames 0, 3, …, 27 (s = 3). For *Jockey* also frames 0–29 (s = 1), to test a longer clip.
- Validation clip for choosing hyperparameters: a segment of Jockey that I will select after inspecting the sequence, taken at both strides. It must not overlap frames 0–29, and it must show content similar to the test clips (the galloping horse with the camera panning). The chosen settings are then used for all three sequences.

**Compute.**
One GPU of the SoC Compute Cluster (an NVIDIA A100), with runs submitted as a Slurm job array. The test runs total 336 fits (288 + 48, see the design table in Section 4), plus the validation sweeps and the synthetic test. In Week 10 I will time one fit at the largest budget. If the full set does not fit in the schedule, I will run *Jockey* and *ShakeNDry* with all four models first, then *YachtRide* with M0 and M1 only.

## 6. Evaluation plan

**Comparison protocol.**
The primary comparison is at equal stored parameters and equal iterations. Following Unit 14, the swept hyperparameters are chosen by PSNR on the validation clip, and each chosen configuration is then reported once on the test clips. Each result is the mean over 3 seeds with its min–max range.

**Metrics**

| Metric | What it tells me |
|---|---|
| Mean PSNR vs stored numbers (one curve per model, per sequence and stride) | The main answer: which model gives the best quality for a given storage |
| PSNR gain over M0 vs measured motion (one point per clip) | Whether the gain grows as motion becomes faster and more curved |
| SSIM | Whether a PSNR gain reflects real structure or only smoother colour |
| PSNR inside vs outside the motion mask | Whether a gain comes from non-uniform motion or from the background |
| Per-frame PSNR and per-frame difference to M0 | Whether a mean hides frames where a model fails |

**Qualitative and diagnostic evidence**
PSNR shows how much a model wins or loses, but not where or why. The images and statistics below show what the errors look like and support the explanations in Section 7.

- The worst frame of each model, with magnified crops of the moving regions (the horse's legs, the dog) and of edges where new background enters.
- Error maps of the same frames.
- The distribution of the fitted accelerations, including the share of Gaussians whose acceleration moves the centre by less than half a pixel over its lifetime, inside and outside the motion mask: a direct measure of storage spent on Gaussians that do not use it.
- The distribution of fitted durations $\sigma t$ and the number of Gaussians active per frame, for M0 against M1.
- M0 against M1 on the synthetic clip, where the true paths are known.
- For M2 and M3, crops that check whether shrinking leaves visible gaps.

**Costs reported alongside quality.**
A quality gain matters only if its cost is acceptable, so I also report what each model costs: stored numbers and bytes, training time per iteration and in total, rendering time per frame, and peak GPU memory.

**What would show the idea did not work.**
I will fix these failure conditions before running the experiments:

- **Acceleration fails** if M1 does not beat M0 by more than 0.3 dB, and by more than the seed-to-seed range, at three or more of the four budgets, on both Jockey and ShakeNDry at stride 3 (the clips with the most curved motion).
- **Shrinking fails** under the same test, applied to M2 against M0.
- If an extension passes only on some sequences or only at stride 3, I will report that it helps only for that kind of motion, not in general.
- If M1 also gains on YachtRide, where the motion is nearly straight, the gain is probably not caused by curved motion, and I will find its cause before drawing a conclusion.
- If a gain appears only inside the motion mask while whole-frame PSNR does not rise, I will report the gain as local and not worth its storage.

## 7. Expected analysis

The final report should answer what happened and why it happened.

**What happened**

- Does acceleration, shrinking, or both raise PSNR at equal storage, and by how much at each budget, on each sequence and stride?
- Does the gain grow from stride 1 to stride 3, and from YachtRide to Jockey and ShakeNDry?
- Does the answer change with the budget? I expect acceleration to matter most at small budgets, where each Gaussian must cover more motion alone, and possibly to lose at large budgets, where short-lived linear Gaussians can relay.
- Is any gain inside the motion mask, outside it, or both?
- What share of the fitted accelerations is close to zero, that is, how much of the extra storage goes unused?
- Does shrinking leave visible gaps, and how sensitive is it to the floor smin?
- Does the 30-frame Jockey clip change the conclusion drawn from the 10-frame clips?
- What do the extensions cost in time and memory?

**Why it happened**

- If acceleration helps, is it because a Gaussian that can curve lives longer and replaces several straight ones? I will compare the fitted durations $\sigma t$ and the number of Gaussians active per frame in M0 and M1.
- If acceleration does not help, is it because the motion does not need curves, or because the optimiser does not learn to use a? The synthetic clip separates the two. If M1 cannot beat M0 even on blobs with known curved paths, the problem is optimization; if it beats M0 there but not on video, the video does not need the extra capacity.
- Where is the extra storage spent? I will compare the share of near-zero accelerations inside and outside the motion mask, and check whether the fitted accelerations agree with the acceleration of the optical flow at each Gaussian's centre.
- Why does the gain change with the budget? I will check whether, at large budgets, M0 covers the moving regions with more and shorter-lived Gaussians, which would confirm the relay explanation.
- Where does shrinking help, and why? I will compare its error reduction with the regions where content enters or is uncovered, such as the left edge of Jockey, and check whether the best $s_{min}$ differs between sequences.

## 8. Extension (if time permits)

The slicing module will support a polynomial trajectory of any degree d, at a cost of $12 + 2(d - 1)$ numbers per Gaussian. If the main experiments finish on schedule, I will compare $d \in \{1, 2, 3, 7\}$ at equal storage on the 30-frame Jockey clip, which tests whether the higher-degree gain in the VeGaS ablation survives once storage is controlled. I will also render slices at times between frames (for example t = 4.5) to check whether high-degree trajectories oscillate between the frames they were fitted to.

If time still permits, I will add stride s = 2, to see whether the gain changes smoothly with the speed of motion, and HoneyBee as a fourth sequence, where many small objects enter and leave the frame.

## 9. Risks and limitations

- Shrinking can leave gaps that neighbouring Gaussians must fill, and the covariance can become ill-conditioned; the floor $s_{min}$ and its sweep address this, together with the course renderer's lower bound of 0.3 pixels on the diagonal of the Cholesky factor.
- The acceleration may destabilise early training; it starts at zero and has its own swept learning rate.
- Each iteration renders every frame of the clip, so the Warp renderer may be slow on the 30-frame clip at the largest budget; the GaussianImage rasteriser is the fallback.
- The test set has 336 fits, which may not all fit in the schedule. I will time one fit in Week 10 and follow the priority order in Section 5 if needed.
- The motion in 10 consecutive frames is close to uniform, so acceleration may have little to gain there; the stride-3 clips and the 30-frame clip are included for this reason.
- Hyperparameters are chosen on Jockey only and may not be best for ShakeNDry and YachtRide.
- The conclusions hold only for these three sequences, these budgets and this resolution. I will not claim that folding is better or worse in general.
- The horse masks are hand-drawn and the other motion masks come from optical flow, so region results are indicative rather than exact.

## 10. Timeline

1. Week 10 (19–25 Oct): submit the proposal; set up the course program on the cluster and write the loader for the three sequences with frame stride; implement M0, reproduce the Unit 16 10-frame fit, and time one fit at the largest budget.
2. Week 11 (26 Oct–1 Nov): implement M1–M3, check gradients, run the synthetic test, measure motion with optical flow, and run the validation sweeps.
3. Week 12 (2–8 Nov): run all test configurations, draw the horse masks and build the flow masks, run the diagnostics for Section 7, and run the extension if on schedule.
4. Week 13 (9–13 Nov): analyse the results, give the presentation, submit the report by Friday 13 November.

## 11. References


- CP4282 course notes, Unit 16: 2D Gaussians for Images and Videos and Unit 14: Compaction of 3DGS.
- W. Smolak-Dyżewska, D. Malarz, K. Howil, J. Kaczmarczyk, M. Mazur, P. Spurek. VeGaS: Video Gaussian Splatting. arXiv:2411.11024, 2024.
- X. Zhang et al. GaussianImage: 1000 FPS Image Representation and Compression by 2D Gaussian Splatting. ECCV 2024. Code.
- A. Mercat, M. Viitanen, J. Vanne. UVG dataset: 50/120fps 4K sequences for video codec analysis and development. ACM MMSys 2020.
- G. Farnebäck. Two-frame motion estimation based on polynomial expansion. SCIA 2003.


## 12. AI-use note

Claude Opus 5.5 (Anthropic) was used to read the course notes and the VeGaS paper, discuss the experimental design, and draft the wording of this proposal. I made and checked the decisions on the research question, the models compared, the sequences and strides, the budgets and the failure criteria.